

```

import { useState, useEffect, useRef } from "react";

// — Anthropic API helper —————
async function callClaude(systemPrompt, userMessage) {
  const res = await fetch("https://api.anthropic.com/v1/messages", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      model: "claude-sonnet-4-20250514",
      max_tokens: 1000,
      system: systemPrompt,
      messages: [{ role: "user", content: userMessage }],
    }),
  });
  const data = await res.json();
  return data.content.map((b) => b.text || "").join("\n");
}

// — Storage helpers —————
const RECIPES_KEY = "familyFavourites";
const HISTORY_KEY = "suggestionHistory";

async function loadRecipes() {
  try {
    const r = await window.storage.get(RECIPES_KEY);
    return r ? JSON.parse(r.value) : [];
  } catch { return []; }
}

async function saveRecipes(recipes) {
  await window.storage.set(RECIPES_KEY, JSON.stringify(recipes));
}

async function loadHistory() {
  try {
    const r = await window.storage.get(HISTORY_KEY);
    return r ? JSON.parse(r.value) : [];
  } catch { return []; }
}

async function saveHistory(history) {
  await window.storage.set(HISTORY_KEY, JSON.stringify(history.slice(-30)));
}

// — Word doc parser (mammoth) —————
async function parseDocx(file) {
  return new Promise((resolve) => {

```

```

const reader = new FileReader();
reader.onload = async (e) => {
  try {
    const mammoth = window.mammoth;
    if (!mammoth) { resolve(null); return; }
    const result = await mammoth.extractRawText({ arrayBuffer: e.target.result });
    resolve(result.value);
  } catch { resolve(null); }
};
reader.readAsArrayBuffer(file);
});
}

// — Parse raw text into recipe objects via Claude —————
async function parseRecipesFromText(rawText) {
  const sys = `You are a recipe parser. Given raw text that may contain a list of
Each recipe object must have:
  "name": string (recipe title),
  "ingredients": string (comma-separated or brief list, can be empty string),
  "instructions": string (brief steps, can be empty string),
  "tags": array of strings (e.g. ["dinner", "chicken", "quick"])
If the text is just a list of recipe names with no details, set ingredients and i
  const raw = await callClaude(sys, `Parse these recipes:\n\n${rawText}`);
  try {
    const clean = raw.replace(/```json|```/g, "").trim();
    return JSON.parse(clean);
  } catch { return []; }
}

// — Suggest from family list —————
async function suggestFromFamily(recipes, recentNames, count = 4) {
  const sys = `You are a helpful family meal planner. Given a list of recipes and
  const msg = `All recipes: ${recipes.map(r => r.name).join(", ")}\nRecently sugg
  const raw = await callClaude(sys, msg);
  try {
    const clean = raw.replace(/```json|```/g, "").trim();
    return JSON.parse(clean);
  } catch { return recipes.slice(0, count).map(r => r.name); }
}

// — Suggest NEW recipes (outside family list) —————
async function suggestNewRecipes(existingNames, count = 4) {
  const sys = `You are a creative chef. Suggest ${count} new recipes that are NOT
  const msg = `Existing recipes to avoid: ${existingNames.join(", ")}\n\nSuggest
  const raw = await callClaude(sys, msg);
  try {
    const clean = raw.replace(/```json|```/g, "").trim();

```

```

    return JSON.parse(clean);
  } catch { return []; }
}

// — Icons —————
const Icon = ({ name, size = 18 }) => {
  const icons = {
    book: <svg width={size} height={size} viewBox="0 0 24 24" fill="none" stroke=
    wand: <svg width={size} height={size} viewBox="0 0 24 24" fill="none" stroke=
    star: <svg width={size} height={size} viewBox="0 0 24 24" fill="none" stroke=
    plus: <svg width={size} height={size} viewBox="0 0 24 24" fill="none" stroke=
    edit: <svg width={size} height={size} viewBox="0 0 24 24" fill="none" stroke=
    trash: <svg width={size} height={size} viewBox="0 0 24 24" fill="none" stroke
    upload: <svg width={size} height={size} viewBox="0 0 24 24" fill="none" strok
    search: <svg width={size} height={size} viewBox="0 0 24 24" fill="none" strok
    x: <svg width={size} height={size} viewBox="0 0 24 24" fill="none" stroke="cu
    save: <svg width={size} height={size} viewBox="0 0 24 24" fill="none" stroke=
    refresh: <svg width={size} height={size} viewBox="0 0 24 24" fill="none" stro
  };
  return icons[name] || null;
};

// — Tag badge —————
const Tag = ({ label }) => (
  <span style={{
    background: "rgba(180,140,100,0.15)", color: "#a0784a",
    borderRadius: 99, padding: "2px 10px", fontSize: 11, fontWeight: 600,
    letterSpacing: "0.04em", textTransform: "uppercase", border: "1px solid rgba(
  }}>{label}</span>
);

// — Recipe Card —————
const RecipeCard = ({ recipe, onEdit, onDelete }) => {
  const [open, setOpen] = useState(false);
  return (
    <div onClick={() => setOpen(!open)} style={{
      background: "rgba(255,252,245,0.9)", border: "1px solid rgba(200,180,140,0.
      borderRadius: 14, padding: "18px 20px", cursor: "pointer",
      transition: "all 0.2s", boxShadow: open ? "0 8px 32px rgba(160,120,74,0.12)
      transform: open ? "translateY(-1px)" : "none"
    }}>
      <div style={{ display: "flex", justifyContent: "space-between", alignItems:
        <div style={{ flex: 1 }}>
          <div style={{ fontFamily: "'Playfair Display', serif", fontSize: 17, fo
            {recipe.name}
          </div>
          <div style={{ display: "flex", gap: 4, flexWrap: "wrap" }}>

```

```

        {(recipe.tags || []).slice(0, 3).map(t => <Tag key={t} label={t} />)}
      </div>
    </div>
    <div style={{ display: "flex", gap: 6 }} onClick={e => e.stopPropagation(
      <button onClick={() => onEdit(recipe)} style={{ background: "none", bor
      <button onClick={() => onDelete(recipe.id)} style={{ background: "none"
    </div>
  </div>
  {open && (recipe.ingredients || recipe.instructions) && (
    <div style={{ marginTop: 14, paddingTop: 14, borderTop: "1px solid rgba(2
      {recipe.ingredients && <div style={{ marginBottom: 10 }}>
        <div style={{ fontSize: 11, fontWeight: 700, color: "#a0784a", letter
        <div style={{ fontSize: 14, color: "#5a4a3a", lineHeight: 1.6 }}>{rec
      </div>}
      {recipe.instructions && <div>
        <div style={{ fontSize: 11, fontWeight: 700, color: "#a0784a", letter
        <div style={{ fontSize: 14, color: "#5a4a3a", lineHeight: 1.6 }}>{rec
      </div>}
    </div>
  )}
</div>
);
};

// — Recipe Form Modal —————
const RecipeModal = ({ recipe, onSave, onClose }) => {
  const [form, setForm] = useState(recipe || { name: "", ingredients: "", instruc
  const [tagInput, setTagInput] = useState((recipe?.tags || []).join(", "));

  const handleSave = () => {
    if (!form.name.trim()) return;
    const tags = tagInput.split(",").map(t => t.trim()).filter(Boolean);
    onSave({ ...form, tags, id: form.id || Date.now().toString() });
  };

  return (
    <div style={{ position: "fixed", inset: 0, background: "rgba(30,20,10,0.5)",
      <div style={{ background: "#fffdf5", borderRadius: 18, padding: 28, width:
        <div style={{ display: "flex", justifyContent: "space-between", alignItem
          <h2 style={{ fontFamily: "'Playfair Display', serif", fontSize: 22, col
          <button onClick={onClose} style={{ background: "none", border: "none",
        </div>
        {[["Recipe Name", "name", "text", true], ["Ingredients", "ingredients", "
          <div key={field} style={{ marginBottom: 16 }}>
            <label style={{ display: "block", fontSize: 12, fontWeight: 700, colo
              {type === "textarea"
                ? <textarea value={form[field]} onChange={e => setForm({ ...form, [

```

```

      : <input value={form[field]} onChange={e => setForm({ ...form, [fie
    </div>
  )})
  <div style={{ marginBottom: 22 }}>
    <label style={{ display: "block", fontSize: 12, fontWeight: 700, color:
    <input value={tagInput} onChange={e => setTagInput(e.target.value)} pla
  </div>
  <div style={{ display: "flex", gap: 10, justifyContent: "flex-end" }}>
    <button onClick={onClose} style={{ padding: "10px 20px", borderRadius:
    <button onClick={handleSave} style={{ padding: "10px 22px", borderRadiu
  </div>
</div>
</div>
);
};

```

```

// — Suggestion Card —————
const SuggestionCard = ({ item, isNew, onAddToFavourites }) => (
  <div style={{
    background: isNew ? "linear-gradient(135deg, rgba(255,248,235,0.95), rgba(255
    border: isNew ? "1.5px solid rgba(200,160,80,0.4)" : "1px solid rgba(200,180,
    borderRadius: 14, padding: "18px 20px",
    boxShadow: isNew ? "0 4px 20px rgba(200,160,80,0.1)" : "0 2px 8px rgba(0,0,0,
  }}>
    <div style={{ display: "flex", alignItems: "flex-start", justifyContent: "spa
      <div style={{ flex: 1 }}>
        {isNew && <div style={{ fontSize: 10, fontWeight: 800, color: "#c0884a",
        <div style={{ fontFamily: "'Playfair Display', serif", fontSize: 17, font
          {isNew ? item.name : item}
        </div>
        {isNew && item.description && <div style={{ fontSize: 13, color: "#7a6a5a
        {isNew && (item.tags || []).length > 0 && (
          <div style={{ display: "flex", gap: 4, flexWrap: "wrap", marginTop: 8 }
            {item.tags.map(t => <Tag key={t} label={t} />)}
          </div>
        )}
      </div>
      {isNew && (
        <button onClick={() => onAddToFavourites(item)} style={{ background: "#a0
      )}
    </div>
  </div>
);

```

```

// — MAIN APP —————
export default function App() {
  const [tab, setTab] = useState("list"); // list | suggest | new

```

```

const [recipes, setRecipes] = useState([]);
const [history, setHistory] = useState([]);
const [loading, setLoading] = useState(true);
const [search, setSearch] = useState("");
const [editRecipe, setEditRecipe] = useState(null);
const [showForm, setShowForm] = useState(false);
const [suggestions, setSuggestions] = useState([]);
const [newSuggestions, setNewSuggestions] = useState([]);
const [suggesting, setSuggesting] = useState(false);
const [importing, setImporting] = useState(false);
const [importMsg, setImportMsg] = useState("");
const fileRef = useRef();

// Load mammoth.js for Word doc parsing
useEffect(() => {
  if (!window.mammoth) {
    const s = document.createElement("script");
    s.src = "https://cdnjs.cloudflare.com/ajax/libs/mammoth/1.6.0/mammoth.brows
    document.head.appendChild(s);
  }
  loadRecipes().then(r => { setRecipes(r); setLoading(false); });
  loadHistory().then(h => setHistory(h));
}, []);

// Load Google Font
useEffect(() => {
  const link = document.createElement("link");
  link.rel = "stylesheet";
  link.href = "https://fonts.googleapis.com/css2?family=Playfair+Display:wght@6
  document.head.appendChild(link);
}, []);

const saveAndUpdate = async (updated) => {
  setRecipes(updated);
  await saveRecipes(updated);
};

const handleSaveRecipe = async (recipe) => {
  const existing = recipes.findIndex(r => r.id === recipe.id);
  const updated = existing >= 0
    ? recipes.map(r => r.id === recipe.id ? recipe : r)
    : [...recipes, recipe];
  await saveAndUpdate(updated);
  setShowForm(false);
  setEditRecipe(null);
};

```

```

const handleDelete = async (id) => {
  if (!confirm("Delete this recipe?")) return;
  await saveAndUpdate(recipes.filter(r => r.id !== id));
};

const handleImport = async (e) => {
  const file = e.target.files[0];
  if (!file) return;
  setImporting(true);
  setImportMsg("Reading document...");
  const text = await parseDocx(file);
  if (!text) { setImportMsg("Could not read file."); setImporting(false); return; }
  setImportMsg("Parsing recipes with AI...");
  const parsed = await parseRecipesFromText(text);
  if (!parsed.length) { setImportMsg("No recipes found."); setImporting(false); return; }
  const withIds = parsed.map((r, i) => ({ ...r, id: `imported-${Date.now()}-${i}` }));
  const merged = [...recipes];
  for (const r of withIds) {
    if (!merged.find(x => x.name.toLowerCase() === r.name.toLowerCase())) merge
  }
  await saveAndUpdate(merged);
  setImportMsg(`✓ Imported ${withIds.length} recipes!`);
  setImporting(false);
  setTimeout(() => setImportMsg(""), 3000);
  e.target.value = "";
};

const handleSuggest = async () => {
  if (!recipes.length) return;
  setSuggesting(true);
  setSuggestions([]);
  const recent = history.slice(-12);
  const names = await suggestFromFamily(recipes, recent);
  setSuggestions(names);
  const newHistory = [...history, ...names];
  setHistory(newHistory);
  await saveHistory(newHistory);
  setSuggesting(false);
};

const handleSuggestNew = async () => {
  setSuggesting(true);
  setNewSuggestions([]);
  const ideas = await suggestNewRecipes(recipes.map(r => r.name));
  setNewSuggestions(ideas);
  setSuggesting(false);
};

```

```

const handleAddNewToFavourites = async (item) => {
  const recipe = { id: Date.now().toString(), name: item.name, ingredients: "",
    await saveAndUpdate([...recipes, recipe]);
    setNewSuggestions(ns => ns.filter(n => n.name !== item.name));
  };

const filtered = recipes.filter(r =>
  r.name.toLowerCase().includes(search.toLowerCase()) ||
  (r.tags || []).some(t => t.toLowerCase().includes(search.toLowerCase()))
);

const navBtn = (id, icon, label) => (
  <button key={id} onClick={() => setTab(id)} style={{
    flex: 1, padding: "10px 4px", background: tab === id ? "#a0784a" : "none",
    border: "none", borderRadius: 10, cursor: "pointer",
    color: tab === id ? "#fff" : "#7a6a5a", fontFamily: "'Lato', sans-serif",
    fontSize: 12, fontWeight: 700, display: "flex", flexDirection: "column", al
    transition: "all 0.18s"
  }}>
    <Icon name={icon} size={18} />
    {label}
  </button>
);

return (
  <div style={{ minHeight: "100vh", background: "linear-gradient(160deg, #fdf8f
    /* Header */
  <div style={{ background: "rgba(255,252,245,0.85)", backdropFilter: "blur(1
    <div style={{ maxWidth: 640, margin: "0 auto" }}>
      <div style={{ display: "flex", justifyContent: "space-between", alignIt
        <div>
          <div style={{ fontSize: 11, fontWeight: 700, color: "#a0784a", lett
            <h1 style={{ fontFamily: "'Playfair Display', serif", fontSize: 28,
          </div>
          <div style={{ textAlign: "right" }}>
            <div style={{ fontSize: 22, fontWeight: 700, color: "#a0784a", font
              <div style={{ fontSize: 11, color: "#9a8a7a" }}>recipes</div>
            </div>
          </div>
        </div>
      <div style={{ display: "flex", gap: 4, background: "rgba(200,180,140,0.
        {navBtn("list", "book", "All Recipes")}
        {navBtn("suggest", "wand", "Suggest")}
        {navBtn("new", "star", "New Ideas")}
      </div>
    </div>
  </div>

```

```

</div>

{/* Body */}
<div style={{ maxWidth: 640, margin: "0 auto", padding: "20px 16px 80px" }}

  {/* — LIST TAB — */}
  {tab === "list" && (
    <div>
      {/* Import + Add */}
      <div style={{ display: "flex", gap: 10, marginBottom: 16 }}>
        <button onClick={() => fileRef.current.click()} disabled={importing
          flex: 1, padding: "11px 16px", background: "rgba(255,252,245,0.9)
          borderRadius: 12, cursor: "pointer", color: "#a0784a", fontWeight
        }}>
          <Icon name="upload" size={16} /> {importing ? "Importing..." : "Imp
        </button>
        <input ref={fileRef} type="file" accept=".docx,.doc" onChange={hand
        <button onClick={() => { setEditRecipe(null); setShowForm(true); }}
          padding: "11px 18px", background: "#a0784a", color: "#fff", borde
          borderRadius: 12, cursor: "pointer", fontWeight: 700, fontSize: 1
        }}>
          <Icon name="plus" size={16} /> Add
        </button>
      </div>

      {importMsg && (
        <div style={{ background: importMsg.startsWith("✓") ? "rgba(80,160,
          {importMsg}
        </div>
      )}

      {/* Search */}
      <div style={{ position: "relative", marginBottom: 16 }}>
        <span style={{ position: "absolute", left: 12, top: "50%", transfor
        <input value={search} onChange={e => setSearch(e.target.value)} pla
      </div>

      {loading ? (
        <div style={{ textAlign: "center", padding: 40, color: "#a0784a" }}
      ) : filtered.length === 0 ? (
        <div style={{ textAlign: "center", padding: 48 }}>
          <div style={{ fontSize: 40, marginBottom: 12 }}><img alt="book icon" data-bbox="728 833 748 848"/></div>
          <div style={{ fontFamily: "'Playfair Display', serif", fontSize:
            <div style={{ fontSize: 14, color: "#9a8a7a" }}>{search ? "Try a
          </div>
        ) : (
          <div style={{ display: "flex", flexDirection: "column", gap: 10 }}>

```

```

        {filtered.map(r => <RecipeCard key={r.id} recipe={r} onEdit={r =>
        </div>
        )}
    </div>
    )}

{ /* — SUGGEST TAB — */
{tab === "suggest" && (
    <div>
        <div style={{ background: "rgba(255,252,245,0.9)", borderRadius: 16,
        <h2 style={{ fontFamily: "'Playfair Display', serif", fontSize: 20,
        <p style={{ fontSize: 14, color: "#7a6a5a", margin: "0 0 16px", lin
        <button onClick={handleSuggest} disabled={suggesting || !recipes.le
        width: "100%", padding: "13px", background: "#a0784a", color: "#f
        borderRadius: 12, cursor: "pointer", fontWeight: 700, fontSize: 1
        opacity: suggesting || !recipes.length ? 0.6 : 1
        }}>
            <Icon name={suggesting ? "refresh" : "wand"} size={17} />
            {suggesting ? "Finding suggestions..." : "Suggest Recipes"}
        </button>
        </div>

        {suggestions.length > 0 && (
            <div>
                <div style={{ fontSize: 11, fontWeight: 700, color: "#a0784a", le
                <div style={{ display: "flex", flexDirection: "column", gap: 10 }
                    {suggestions.map((s, i) => <SuggestionCard key={i} item={s} isN
                </div>
                <div style={{ marginTop: 14, fontSize: 13, color: "#9a8a7a", text
                </div>
            )}

            {!recipes.length && (
                <div style={{ textAlign: "center", padding: 30, color: "#9a8a7a", f
            )}
        </div>
    )}

{ /* — NEW IDEAS TAB — */
{tab === "new" && (
    <div>
        <div style={{ background: "linear-gradient(135deg, rgba(255,248,235,0
        <h2 style={{ fontFamily: "'Playfair Display', serif", fontSize: 20,
        <p style={{ fontSize: 14, color: "#7a6a5a", margin: "0 0 16px", lin
        <button onClick={handleSuggestNew} disabled={suggesting} style={{
        width: "100%", padding: "13px", background: "linear-gradient(135d
        borderRadius: 12, cursor: "pointer", fontWeight: 700, fontSize: 1

```

```

        opacity: suggesting ? 0.6 : 1
      }}>
      <Icon name={suggesting ? "refresh" : "star"} size={17} />
      {suggesting ? "Discovering ideas..." : "Find New Ideas"}
    </button>
  </div>

  {newSuggestions.length > 0 && (
    <div>
      <div style={{ fontSize: 11, fontWeight: 700, color: "#c0884a", le
      <div style={{ display: "flex", flexDirection: "column", gap: 10 }
        {newSuggestions.map((s, i) => <SuggestionCard key={i} item={s}
      </div>
    </div>
  )}
</div>
)}
</div>

{/* Recipe Form Modal */}
{showForm && (
  <RecipeModal
    recipe={editRecipe}
    onSave={handleSaveRecipe}
    onClose={() => { setShowForm(false); setEditRecipe(null); }}
  />
)}
</div>
);
}

```